Predicting Explanation Error Without Causal Ground Truth: A Held-Out-SCM Study of SHAP and LIME

Bhuvi S. Katti
PES1UG24AM418
Dept. of CSE (AI&ML)
PES University, Bengaluru

Mythri S
PES1UG24AM419
Dept. of CSE (AI&ML)
PES University, Bengaluru

Poojitha L. B. R.
PES1UG24AM384
Dept. of CSE (AI&ML)
PES University, Bengaluru

Diran Somanna
PES1UG24AM364
Dept. of CSE (AI&ML)
PES University, Bengaluru

***Abstract*—Feature attribution methods such as SHAP and LIME are often read as showing which features cause an outcome, even though they only describe a trained model. In real data the true causes are unknown, so it is not possible to check whether an explanation is misleading. In this project we ask whether the error of an explanation, measured against the true causal effect, can be predicted using only information that is available without knowing the true causes. We build a benchmark of 300 synthetic structural causal models from five graph families, where the true effects are known. We train XGBoost and MLP models, run five explainers, and split each explanation's error into three parts: the estimand gap (SHAP and the causal effect measure different things), the model misfit, and the explainer's own approximation error. The estimand gap is the largest part: even with a perfect model and exact SHAP, the importance shares are 0.31 (L_2) away from the true ones, while KernelSHAP adds less than 0.01. In chain-shaped graphs, 38% of the true effect comes from indirect causes that SHAP gives zero credit to. Using 39 signals that do not need the true causes, a gradient-boosted meta-model predicts the share error with a Spearman correlation of 0.65 on unseen models and 0.52 (AUROC 0.76) on graph families it was not trained on, and beats all simple baselines ($p < 0.001$). When each part of the error is predicted separately, the explainer error ($\rho = 0.91$) and model misfit (0.70) are easy to predict, but the estimand gap (0.34) is not, and the error in feature ranking (0.22) is hardly predictable. This means such a check can warn when the model or the explainer is the problem, but not when SHAP itself answers a different question from the causal one.**

***Index Terms*—explainable AI, SHAP, LIME, structural causal models, feature attribution, causal inference, meta-learning, benchmark**

I. Introduction

Feature attribution methods such as SHAP [1] and LIME [2] are widely used to explain the predictions of machine learning models. In practice, their outputs are often read in a causal way: if a feature gets a high attribution, people assume that the feature *causes* the outcome, and that changing it will change the outcome. However, this is not what these methods compute. SHAP measures how much a feature changes the *model's* prediction compared to a reference input [3], and LIME fits a simple linear model around one prediction. Neither method knows how the outcome would respond to an actual intervention $P(Y \mid \mathrm{do}(X_j))$.

There are well-known reasons why the two can differ. A model can rely on a feature that is only correlated with the true cause, an effect that passes through a middle variable (a mediator) is credited to that middle variable instead of the original cause, and approximate explainers add their own sampling error [4]. The main problem is that in real data we do not know the true causes, so we cannot check whether an explanation is misleading.

In this project we ask a simpler and more practical question: *can we predict how wrong an explanation is, using only information that is available without knowing the true causes?* For example, if explanations tend to be wrong when features are strongly correlated, when the model fits poorly, or when different explainers disagree, then these signals could be used to warn a user that an explanation should not be trusted.

To study this, we use synthetic data where the true causal structure is known. We generate structural causal models (SCMs), compute the true effect of each feature, train models on data sampled from them, and run five explainers. We then measure how far each explanation is from the true effect. Next, we compute signals that do *not* need the true structure (we call these *observables*), and train a second model, a *meta-model*, to predict the explanation error from them. Finally, we test whether the meta-model still works on types of causal graphs that it has never seen during training.

Our main contributions are:

1) A benchmark of 300 SCMs from five graph families, with exact causal effects and exact Shapley values of the true function. We split each explanation's error into three parts: the *estimand gap*, the *model misfit* and the *explainer error*.
2) Measurements of how large each part of the error is, for XGBoost and MLP models explained by exact SHAP, TreeSHAP, KernelSHAP, LIME and permutation importance.
3) A meta-model that predicts explanation error from 39 observables, compared against seven simple baselines, and tested on unseen SCMs and on unseen graph families.

All code, data and the notebook that reproduces the tables and figures are available at https://github.com/stacktrace-bhuvi/datasphere.

II. Related Work

Attribution methods. SHAP [1] is based on Shapley values from game theory. KernelSHAP estimates them by sampling subsets of features, and TreeSHAP computes them exactly for tree models. LIME [2] fits a small linear model around the input being explained. Permutation importance [6] measures how much the model's error increases when one feature is shuffled; it gives one score per feature for the whole dataset instead of one per prediction.

Attributions and causality. Janzing et al. [3] argue that SHAP should use the *interventional* (marginal) version of Shapley values, because the conditional version can give credit to features the model does not use. Still, even interventional SHAP describes the model, not the real process that generated the data. The two only agree when the model's inputs are exactly the direct causes of the outcome. Heskes et al. [7] and Frye et al. [8] propose versions of Shapley values that use a known causal graph. We do not assume the graph is known. Instead, we measure how far standard attributions are from the true effect, and ask if this distance can be predicted.

Evaluating explanations. OpenXAI [5] compares explainers using faithfulness and stability measured against the *model*. Krishna et al. [4] show that different explainers often rank features differently (the *disagreement problem*), and Alvarez-Melis and Jaakkola [9] study how much explanations change when the input changes slightly. In our work, disagreement and stability are not used to judge explainers. We use them as *inputs* to a model that predicts error,

and we measure error against the true causal effect instead of the model.

III. Problem Setup

A. Structural causal models and the causal effect

An SCM over features $X_1, \dots, X_d$ and an outcome Y is made of a directed acyclic graph (DAG) G and one equation per node, $X_v = f_v(\mathrm{pa}(v)) + \varepsilon_v$, where $\mathrm{pa}(v)$ are the parents of v and ε_v is independent noise. Y has no children, and its parents are features. Normal (observational) data is sampled from these equations. An intervention $\mathrm{do}(X_j = x)$ replaces the equation of X_j with the fixed value x, and the change flows to all nodes downstream of X_j.

The true effect we compare against is the *total interventional effect* of each feature:

$$\tau_j = \mathbb{E}_{x\sim P(X_j)}\Big[\tfrac{1}{2}\big|\,\mathbb{E}[Y \mid \mathrm{do}(X_j{=}x{+}\sigma_j)] - \mathbb{E}[Y \mid \mathrm{do}(X_j{=}x{-}\sigma_j)]\big|\Big], \tag{1}$$

where σ_j is the standard deviation of X_j. In simple words, τ_j is how much Y changes on average when we push X_j up or down by one standard deviation. This matches how people usually read an attribution. It also includes indirect effects that pass through other features.

B. What attributions measure

Let $f^*(x) = \mathbb{E}[Y \mid x]$ be the true outcome function without noise (it only depends on the parents of Y), and let f be a trained model. For any function g, the interventional Shapley value of feature j for input x, with background data B, is

$$\phi_j(g; x) = \sum_{S \subseteq [d]\setminus\{j\}} \tfrac{|S|!\,(d-|S|-1)!}{d!}\big[v_g(S\cup\{j\}) - v_g(S)\big], \tag{2}$$

where $v_g(S) = \mathbb{E}_{x_B\sim B}[g(x_S, x_{B,\bar{S}})]$ is the average prediction when the features in S are taken from x and the remaining features come from the background data. To get one number per feature, we average the absolute value over all explained inputs, $\bar{\phi}_j(g) = \frac{1}{N}\sum_i |\phi_j(g; x^{(i)})|$, and compare the vector $\bar{\boldsymbol{\phi}}$ with $\boldsymbol{\tau}$.

C. Splitting the error into three parts

Let $\hat{E}_j$ be the attribution that an explainer actually outputs for the trained model. The difference between the true effect and the explanation can be written as a sum of three differences:

$$\underbrace{\boldsymbol{\tau} - \hat{E}}_{\text{total}} = \underbrace{\boldsymbol{\tau} - \bar{\boldsymbol{\phi}}(f^*)}_{\text{estimand gap}} + \underbrace{\bar{\boldsymbol{\phi}}(f^*) - \bar{\boldsymbol{\phi}}(f)}_{\text{model misfit}} + \underbrace{\bar{\boldsymbol{\phi}}(f) - \hat{E}}_{\text{explainer error}}. \tag{3}$$

The *estimand gap* is the error that remains even with a perfect model and an exact explainer. It exists because SHAP and the causal effect measure different things. The *model misfit* is the change caused by the trained model being different from the true function. The *explainer error* is the approximation error of KernelSHAP, LIME or permutation importance compared to exact SHAP of the same model. The vectors add up exactly, but their sizes (norms) do not, so we report each part separately.

$\boldsymbol{\tau}$ is in units of the outcome while $\bar{\boldsymbol{\phi}}$ is in units of model contribution, so comparing them directly is unfair. We therefore measure every part in three ways: (i) normalised L_2 distance, $\|a-b\|_2/\|a\|_2$; (ii) *share error*, the L_2 distance between importance shares $s(v) = |v|/\|v\|_1$, which ignores scale; and (iii) *rank error*, $1 -$ Kendall τ_b, which only looks at the order of features. The two main targets we try to predict are the share error $\|s(\boldsymbol{\tau}) - s(\hat{E})\|_2$ and the rank error $1 - \tau_b(\boldsymbol{\tau}, \hat{E})$.

IV. Benchmark Construction

A. Graph families and mechanisms

We generate SCMs from five graph families, each chosen to create a different kind of problem for explainers: *chain-mediation* ($X_1 \to X_2 \to \cdots \to Y$ with side branches), *confounded fork* ($X_1 \to X_2$ and $X_1 \to Y$), *feature collider* ($X_1 \to X_3 \leftarrow X_2$, $X_3 \to Y$), *Erdős–Rényi DAG* (random edges with probability 0.35) and *layered* (three layers of nodes). The number of features d goes from 5 to 10 and is balanced within each family. Each node has a non-linear equation ($\tanh$, $\sin$, quadratic or ReLU) with probability 0.5, an interaction term between two parents with probability 0.2, and is linear otherwise. Weights are scaled by $1/\sqrt{k}$ for k parents, and values are clipped to $[-10, 10]$. An SCM is rejected and re-generated if too many values get clipped; in the final benchmark no values are clipped.

In our first version (Review 1), the noise on Y was too large compared to the signal: the true function f^* explained only about 33% of the variance of Y ($R^2 \approx 0.33$). We added a parameter that scales down only the noise of Y (`outcome_noise_scale` $= 0.4$). This raises the average R^2 of f^* to 0.67 (range 0.23–0.94) without changing the distribution of the features.

The final benchmark has 60 SCMs per family (300 in total). Each SCM has 2000 samples, split 60/20/20 into train, validation and test sets. The layered family is kept aside as a held-out family; the other four are used for training.

B. Ground truth

We estimate τ_j in (1) by simulation. We pick 50 values of X_j from its distribution, and for each value we simulate the two interventions using the same 5000 noise samples, so that the noise cancels out. We average over the noise *before* taking the absolute value, because taking the absolute value first would overestimate the effect for non-linear equations. The whole estimate is repeated five times and its standard error is stored. For the true function f^* we compute exact Shapley values by going through all 2^d feature subsets, on a fixed set of 100 test rows (to explain) and 50 training rows (as background). Every explainer uses these same rows, so all terms in (3) can be compared directly. We checked that the exact values satisfy the standard Shapley properties (efficiency, linearity, dummy) up to 10^{-8}.

V. Models, Explainers and Error Measurement

On every SCM we train two models with three random seeds each: XGBoost [11] (depth 4, up to 500 trees with early stopping on the validation set) and a multilayer perceptron (MLP) with two hidden layers of 64 units (Adam optimiser, early stopping on the same validation set).

We run five explainers on the same explain and background rows: (i) *exact* SHAP of the trained model, computed by brute force over all feature subsets, which gives the $\bar{\boldsymbol{\phi}}(f)$ term; (ii) TreeSHAP (XGBoost only; it matches exact SHAP to 10^{-4}); (iii) KernelSHAP with $\max(2d{+}2, 2^d/4)$ sampled subsets, so it is always an approximation; (iv) LIME with 1000 samples per input; and (v) permutation importance on the test set (10 repeats). Permutation importance gives scores in units of error increase, so we only compare it using the share and rank errors. For each explanation, the error table is built by combining it with the exact SHAP output for the same SCM, model and seed.

VI. Observables

For every explanation (one SCM, model, explainer and seed) we compute 39 observables. None of them use $\boldsymbol{\tau}$, the true function or the graph, so all of them can be computed on real data.

Data level (training set only): the largest and average absolute correlation between features, the fraction of feature pairs with $|r| > 0.3$, the largest and average variance inflation factor (VIF), the largest and average mutual information between features, the largest and average correlation and mutual information between each feature and Y, the number of training rows and d.

Model level: test and validation error, test and train R^2 and the gap between them, how spread out the predictions are compared to Y, the shape of the test residuals (kurtosis), and the correlation between the predictions of XGBoost and the MLP (*model agreement*).

Explainer level: how concentrated the attributions are (Gini coefficient, share of the top feature, effective number of features), whether the sign of each feature's attribution is consistent across inputs, how stable the explainer is across random seeds, how much it disagrees with the other explainers on the same model, how much the same explainer disagrees between XGBoost and the MLP, and how much the attributions change when the inputs are slightly perturbed (Gaussian noise of 0.05 standard deviations).

The name of the model and the explainer are also given to the meta-model, since a user always knows which ones they used.

VII. Evaluation Method

Each row of our dataset is one explanation. The inputs are the observables and the target is its error. We train three meta-models: gradient-boosted trees (GBM, 200 trees of depth 3), a random forest and ridge regression. We compare them with seven simple baselines that represent checks a user might already do: always predicting the average error (constant), using feature correlation only, maximum VIF only, test error only, sample size only, explainer disagreement only, and explainer stability only. Each baseline is a ridge regression on the listed columns.

Rows from the same SCM share the same data, so they are not independent. Therefore we never put rows from one SCM in both the training and the test set. We use three evaluation setups. *Grouped CV*: five-fold cross-validation where each fold holds out whole SCMs, keeping the families balanced (tests unseen SCMs from known families). *Leave-one-graph-family-out (LOGFO)*: train on four families and test on the fifth, repeated for all five families (tests unseen families). *Held-out split*: train on the four training families and test on the layered family.

We report the Spearman correlation ρ between predicted and actual error, R^2, the mean absolute error (MAE), and the AUROC for detecting explanations whose error is above the training median. To check whether the meta-model is really better than a baseline, we use a bootstrap that resamples whole SCMs (2000 times) and a Wilcoxon signed-rank test on the average error per SCM. We also measure performance separately for each (model, explainer) pair, which tests whether the meta-model can rank SCMs even when the explanation method is fixed.

VIII. Results

All numbers are averages over the 300 SCMs and three seeds unless stated otherwise. The held-out family (layered) is marked with †. XGBoost and the MLP reach an average test R^2 of 0.63 and 0.62 (range 0.10–0.93), compared to 0.67 (range 0.23–0.94) for the true function f^*. So the models learn the outcome reasonably well, but not perfectly.

TABLE I

The three parts of the error, averaged over SCMs and seeds. Share: L_2 distance between importance shares. Rank: $1-$Kendall τ_b. Top-k: fraction of true top features found.

		Share space				Rank	Top-k
Model	Explainer	Gap	Misfit	Expl.	Total	Total	
XGBoost	Exact SHAP	0.31	0.09	0.00	0.26	0.43	0.90
XGBoost	TreeSHAP	0.31	0.09	0.00	0.26	0.43	0.90
XGBoost	KernelSHAP	0.31	0.09	0.00	0.26	0.43	0.89
XGBoost	LIME	0.31	0.09	0.05	0.28	0.44	0.89
XGBoost	Permutation	0.31	0.09	0.15	0.35	0.43	0.90
MLP	Exact SHAP	0.31	0.14	0.00	0.23	0.43	0.91
	KernelSHAP	0.31	0.14	0.01	0.23	0.42	0.91
	LIME	0.31	0.14	0.07	0.27	0.45	0.89
	Permutation	0.31	0.14	0.17	0.32	0.43	0.90

TABLE II

Estimand gap for each graph family (perfect model, exact SHAP), mean $\pm$ standard deviation over SCMs, and total rank error of KernelSHAP and LIME on the trained models. †held-out family.

Family	Gap (rank)	Gap (share)	KernelSHAP	LIME
Chain	0.37 ± 0.10	0.41 ± 0.19	0.52	0.51
Fork	0.34 ± 0.22	0.19 ± 0.08	0.39	0.41
Collider	0.34 ± 0.11	0.36 ± 0.13	0.44	0.48
Random DAG	0.26 ± 0.12	0.26 ± 0.15	0.36	0.38
Layered†	0.38 ± 0.20	0.32 ± 0.15	0.43	0.47

A. Where the explanation error comes from

Table I and Fig. 1 show the three parts of the error. We make three observations, which hold for both models and all explainers.

The estimand gap is the largest part. Even with the true function and exact Shapley values, the importance shares are 0.31 (L_2) away from the true causal shares, and the rank error is 0.37. Model misfit adds 0.09 (XGBoost) to 0.14 (MLP). The explainer's own error is very small for KernelSHAP (0.003–0.007), which shows that its sample size of $2^d/4$ subsets is enough. It is moderate for LIME (0.06). Only permutation importance, which works on the whole dataset and uses the model's loss, has an explainer error (0.15–0.17) as large as the model misfit.

A better explainer does not fix the causal ranking. The rank error against $\boldsymbol{\tau}$ is 0.43 for exact SHAP of the trained model, 0.43 for TreeSHAP and KernelSHAP, 0.44–0.46 for LIME and 0.43 for permutation importance. All methods find 89–91% of the true top features. Using an exact explainer instead of an approximate one only reduces the third term of (3), and the ranking stays almost the same.

The parts do not simply add up. The MLP has a larger model misfit than XGBoost, but a smaller total share error (0.23 vs 0.26 with exact SHAP). The MLP learns a smoother function and spreads some credit onto correlated upstream features. The true function ignores these features, but they do have a causal effect, so this moves the explanation *closer* to the true effect. This is why we report the size of each part separately.

B. The estimand gap depends on the graph structure

Table II and Fig. 2 show the estimand gap for each family. It is largest for chain-mediation graphs (0.41 share error). In these graphs, on average 38% of the true effect comes from features that affect Y only through other features. The true function does not use these

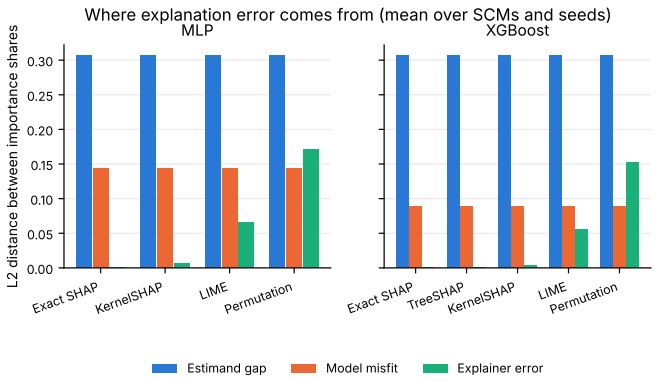


Fig. 1. The three parts of the share error for each explainer and model, averaged over SCMs and seeds. The estimand gap (blue) is the same for all explainers; the explainer's own error (green) is the smallest part except for permutation importance.

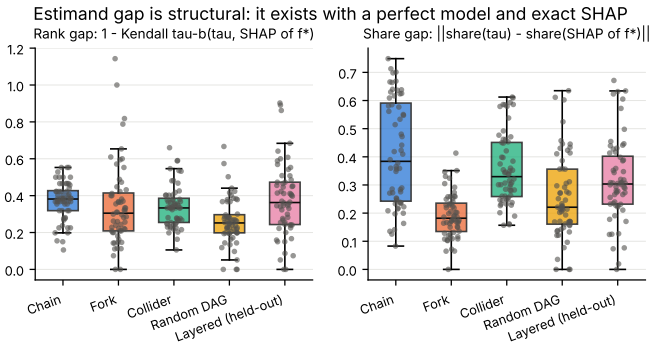


Fig. 2. Estimand gap for each graph family, one point per SCM (rank error on the left, share error on the right). The gap exists even with a perfect model and an exact explainer.

features directly, so exact SHAP gives them zero credit; KernelSHAP and LIME on the trained models give them 15% and 11%. The confounded-fork family has the opposite problem: features that are not parents of Y have only 8% of the true effect but get 10% of the credit from KernelSHAP, because the trained models use features that are only correlated with the true cause. The collider (0.36) and layered[†] (0.32) families come next, and random DAGs, which have many direct parents of Y, have the smallest gap (0.26). The rank error follows a similar order. All 300 SCMs have a non-zero rank gap.

C. Predicting the error without the true causes

Tables III and IV show how well the error can be predicted under the three evaluation setups. Table V and Fig. 3 show the LOGFO results for each held-out family.

The share error can be predicted, also on new families. The GBM meta-model reaches a Spearman correlation of 0.65 ($R^2 = 0.53$) on unseen SCMs from known families, 0.52 ($R^2 = 0.33$; 95% bootstrap interval $[0.43, 0.62]$) under LOGFO, and 0.48 on the held-out layered family. The AUROC for detecting high-error explanations is 0.82, 0.76 and 0.75. The random forest (0.54) and ridge (0.47) meta-models give similar results under LOGFO, so the result does not depend on one particular model. For each held-out family (Table V), the meta-model is the best method, with ρ between 0.29 (fork) and 0.81 (collider). The best simple baseline is test error ($\rho = 0.36$),

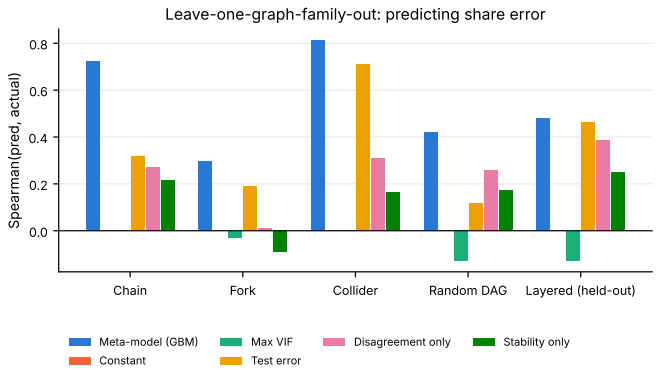


Fig. 3. LOGFO: Spearman correlation between predicted and actual share error for each held-out family, for the GBM meta-model and the simple baselines.

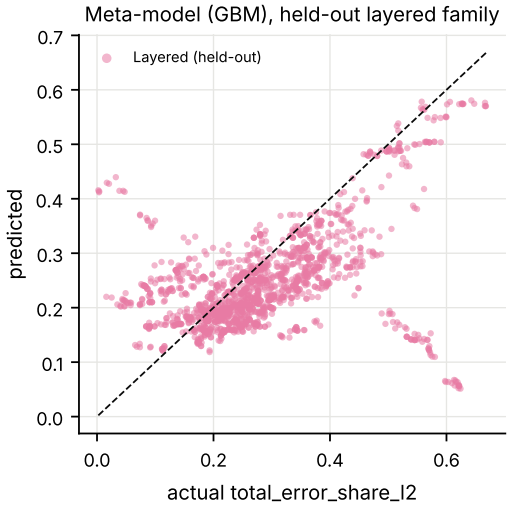


Fig. 4. Predicted vs actual share error on the held-out layered family, for the GBM meta-model trained on the other four families.

followed by explainer disagreement (0.23). Correlation, maximum VIF and the constant baseline do not work at all.

The rank error is much harder to predict. For the rank error, the meta-model reaches $\rho = 0.34$ on known families, but only 0.22 under LOGFO and 0.20 on the layered family, with R^2 close to zero. Only the disagreement and stability baselines show some signal (0.19 and 0.15), and the meta-model is not significantly better than them (Section VIII-E). With at most ten features, the rank error can only take a few values and depends mostly on which features get exactly zero credit. As the next section shows, the observables cannot see this.

D. Which part of the error can be predicted

To understand this better, we trained the meta-model to predict each part of (3) separately (Table VI and Fig. 5). The *explainer error* is very easy to predict ($\rho = 0.91$, $R^2 = 0.98$ under LOGFO). Disagreement between explainers alone already gives $\rho = 0.65$. The *model misfit* is also predicted well ($\rho = 0.70$, $R^2 = 0.59$). Here, test error alone gives only 0.15 and disagreement alone 0.37, so combining several signals helps; the most useful inputs are the disagreement between XGBoost and MLP explanations, the model

TABLE III

PREDICTING THE SHARE ERROR OF AN EXPLANATION FROM OBSERVABLES: SPEARMAN ρ (PREDICTED VS ACTUAL), R^2, AND AUROC FOR DETECTING EXPLANATIONS WITH ABOVE-MEDIAN ERROR, FOR THE THREE EVALUATION SETUPS.

	Grouped CV			LOGFO			Held-out layered		
Method	ρ	R^2	AUC	ρ	R^2	AUC	ρ	R^2	AUC
Meta: GBM	0.65	0.53	0.82	0.52	0.32	0.76	0.48	0.10	0.75
Meta: RF	0.67	0.56	0.83	0.54	0.38	0.76	0.45	0.12	0.74
Meta: ridge	0.59	0.47	0.78	0.47	0.22	0.71	0.48	0.17	0.67
Constant	-0.06	0.00	0.47	-0.35	-0.07	0.34	0.00	-0.01	0.50
Correlation only	0.05	0.00	0.52	-0.23	-0.27	0.39	0.11	-0.02	0.54
Max VIF	0.04	0.00	0.52	-0.29	-0.09	0.36	-0.13	-0.04	0.42
Test error	0.40	0.18	0.70	0.36	0.13	0.67	0.46	0.21	0.72
Sample size	0.19	0.05	0.59	0.07	-0.03	0.53	0.07	-0.06	0.53
Disagreement only	0.34	0.14	0.66	0.23	0.07	0.61	0.38	0.14	0.67
Stability only	0.23	0.08	0.60	0.12	0.03	0.54	0.25	0.07	0.59

TABLE IV

PREDICTING THE RANK ERROR ($1 - \tau_b$) OF AN EXPLANATION FROM OBSERVABLES: SPEARMAN ρ (PREDICTED VS ACTUAL), R^2, AND AUROC FOR DETECTING EXPLANATIONS WITH ABOVE-MEDIAN ERROR, FOR THE THREE EVALUATION SETUPS.

	Grouped CV			LOGFO			Held-out layered		
Method	ρ	R^2	AUC	ρ	R^2	AUC	ρ	R^2	AUC
Meta: GBM	0.34	0.12	0.67	0.22	0.02	0.61	0.20	-0.01	0.59
Meta: RF	0.31	0.09	0.65	0.20	0.02	0.61	0.20	-0.04	0.60
Meta: ridge	0.33	0.12	0.66	0.23	0.02	0.62	0.20	-0.02	0.61
Constant	-0.11	-0.01	0.44	-0.24	-0.03	0.40	0.00	0.00	0.50
Correlation only	0.01	-0.01	0.51	-0.21	-0.06	0.41	-0.20	-0.08	0.40
Max VIF	-0.11	-0.01	0.44	-0.25	-0.05	0.38	-0.14	-0.01	0.43
Test error	0.07	0.00	0.54	-0.06	-0.05	0.48	0.02	0.00	0.48
Sample size	0.07	0.00	0.54	-0.05	-0.03	0.49	0.16	0.02	0.61
Disagreement only	0.22	0.06	0.60	0.18	0.03	0.59	0.14	0.02	0.57
Stability only	0.19	0.04	0.60	0.15	0.02	0.58	0.17	0.03	0.59

TABLE V

LOGFO, SHARE ERROR: SPEARMAN ρ (AUROC IN BRACKETS) FOR EACH HELD-OUT FAMILY. EACH COLUMN USES A MODEL TRAINED ON THE OTHER FOUR FAMILIES.

Method	Chain	Fork	Collider	Random	Layered
Meta: GBM	0.72 (0.84)	0.29 (0.69)	0.81 (0.88)	0.42 (0.77)	0.48 (0.75)
Meta: RF	0.78 (0.88)	0.32 (0.70)	0.83 (0.89)	0.42 (0.77)	0.45 (0.74)
Meta: ridge	0.70 (0.86)	0.25 (0.66)	0.79 (0.85)	0.31 (0.71)	0.48 (0.67)
Constant	0.00 (0.50)	0.00 (0.50)	0.00 (0.50)	0.00 (0.50)	0.00 (0.50)
Correlation only	0.36 (0.66)	0.12 (0.55)	0.37 (0.63)	-0.01 (0.53)	0.10 (0.54)
Max VIF	0.00 (0.49)	-0.03 (0.49)	0.00 (0.49)	-0.13 (0.43)	-0.13 (0.42)
Test error	0.32 (0.61)	0.19 (0.65)	0.71 (0.82)	0.12 (0.58)	0.46 (0.72)
Sample size	0.41 (0.68)	0.16 (0.58)	0.48 (0.68)	0.06 (0.53)	0.07 (0.53)
Disagreement only	0.27 (0.68)	0.01 (0.50)	0.31 (0.64)	0.26 (0.65)	0.38 (0.67)
Stability only	0.21 (0.61)	-0.09 (0.42)	0.16 (0.57)	0.17 (0.62)	0.25 (0.59)

TABLE VI

HOW WELL EACH PART OF THE ERROR CAN BE PREDICTED: LOGFO SPEARMAN ρ (R^2 IN BRACKETS) WHEN EACH PART OF (3), AND THE TWO TOTALS, IS THE TARGET.

Target	Meta: GBM	Test error	Disagreement	Stability	Max VIF
Explainer error	0.91 (0.98)	0.03 (0.00)	0.65 (0.78)	0.33 (0.05)	-0.01 (0.00)
Model misfit	0.70 (0.59)	0.15 (0.02)	0.37 (0.12)	0.44 (0.25)	-0.16 (-0.04)
Estimand gap	0.34 (0.11)	0.34 (0.11)	0.16 (0.02)	0.13 (0.01)	-0.37 (-0.14)
Total (share)	0.52 (0.32)	0.36 (0.13)	0.23 (0.07)	0.12 (0.03)	-0.29 (-0.09)
Total (rank)	0.22 (0.02)	-0.06 (-0.05)	0.18 (0.03)	0.15 (0.02)	-0.25 (-0.05)

type, and how concentrated the attributions are. The *estimand gap*, which is the largest part, is the hardest to predict ($\rho = 0.34$, $R^2 = 0.11$), and the meta-model is no better than test error alone (0.34).

The explanation is simple. The two parts that a user can fix (by training a better model or using a better explainer) are the parts that the observables can detect. The largest part depends on which correlated features are true causes, which are indirect causes and which are only correlated. Nothing in the data, the trained model or the explanations shows this difference. Since this part is the largest, the total share error is only moderately predictable, and the rank error, which is almost entirely this part, is hardly predictable.

E. Is the improvement over the baselines significant?

Table VII compares the meta-model with each baseline for the share error. Under LOGFO the meta-model is better than every baseline. Compared to the best one (test error), the Spearman correlation improves by 0.17 (95% interval $[0.08, 0.25]$), the MAE improves by 0.019 ($[0.012, 0.026]$), the Wilcoxon test gives $p < 0.001$, and the meta-model has a lower error on 65% of SCMs. For the other baselines the improvements are larger, and all intervals are above zero. On the layered family alone, the meta-model is better than the constant, correlation, VIF, sample size and stability baselines ($p \leq 0.05$), but not significantly better than test error ($p = 0.62$)

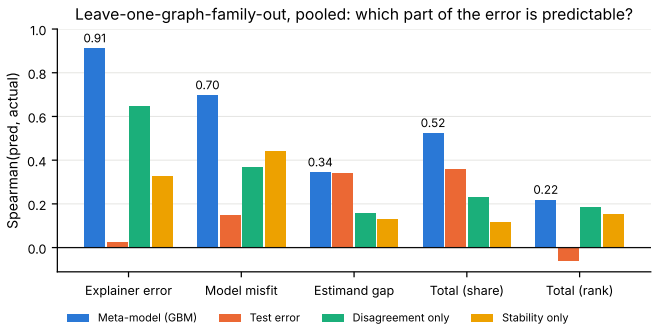


Fig. 5. LOGFO Spearman correlation when each part of the error is the prediction target. The model and explainer parts can be predicted; the estimand gap mostly cannot.

TABLE VII

IMPROVEMENT OF THE GBM META-MODEL OVER EACH BASELINE (SHARE ERROR). BOOTSTRAP OVER SCMS (2000 SAMPLES): AVERAGE IMPROVEMENT IN SPEARMAN ρ AND IN MAE WITH 95% INTERVAL; WILCOXON TEST p-VALUE ON PER-SCM ERRORS; PERCENTAGE OF SCMS WHERE THE META-MODEL HAS THE LOWER ERROR.

Protocol	vs.	$\Delta\rho$ [95% CI]	ΔMAE [95% CI]	p_W	
LOGFO	Constant	0.87 [0.74, 1.00]	0.033 [0.024, 0.042]	<0.001	60%
	Correlation only	0.75 [0.63, 0.86]	0.043 [0.035, 0.052]	<0.001	62%
	Max VIF	0.81 [0.68, 0.94]	0.034 [0.025, 0.043]	<0.001	50%
	Test error	0.17 [0.08, 0.25]	0.019 [0.013, 0.026]	<0.001	52%
	Sample size	0.45 [0.34, 0.57]	0.030 [0.022, 0.038]	<0.001	58%
	Disagreement only	0.29 [0.20, 0.39]	0.024 [0.016, 0.031]	<0.001	65%
	Stability only	0.41 [0.31, 0.50]	0.027 [0.019, 0.035]	<0.001	
Layered	Constant	0.48 [0.22, 0.71]	0.017 [0.001, 0.035]	0.050	
	Correlation only	0.37 [0.07, 0.65]	0.016 [0.000, 0.034]	0.042	
	Max VIF	0.61 [0.22, 0.95]	0.019 [0.003, 0.037]	0.018	
	Test error	0.02 [-0.28, 0.28]	0.004 [-0.015, 0.020]	0.622	
	Sample size	0.41 [0.04, 0.77]	0.018 [-0.002, 0.038]	0.092	
	Disagreement only	0.10 [-0.20, 0.36]	0.009 [-0.006, 0.023]	0.119	
	Stability only	0.24 [-0.05, 0.50]	0.013 [-0.001, 0.028]	0.022	

or disagreement ($p = 0.12$). With only 60 SCMs in one family, the intervals are wide. For the rank error, the meta-model is not significantly better than the disagreement and stability baselines in any setup.

F. Which observables are most useful

Table VIII shows which observables the meta-model depends on most, measured on the layered family, and Fig. 6 shows the strongest individual relationships. Two groups of observables are useful. The first is *concentration*: explanations that give most of the credit to a few features are far from the true shares (effective number of features $\rho = -0.58$, top feature share $+0.57$, Gini $+0.56$). This is because the true effect is spread along chains and forks, while f^*, and any model that learns it, focuses on the direct parents of Y. The second group is *how strongly the features relate to Y, and how well the model fits*: the average feature–outcome correlation is the most important input (0.19), and mutual information with Y (0.42), test R^2 ($+0.40$) and model agreement ($+0.39$) are also strongly related. Note that a better fit means a *larger* error here: well-fit outcomes usually have a few strong direct parents, which is exactly where SHAP focuses its credit while the true effect is spread upstream. Disagreement between explainers ($\rho = 0.34$) and stability add smaller contributions. Maximum VIF, which is often checked in practice, is almost useless on its own.

TABLE VIII

MOST IMPORTANT OBSERVABLES IN THE GBM META-MODEL: DROP IN R^2 ON THE LAYERED FAMILY WHEN THE COLUMN IS SHUFFLED (MEAN $\pm$ STANDARD DEVIATION OVER 10 REPEATS).

Observable	ΔR^2
`mean_abs_corr_xy`	0.188 $\pm$ 0.022
`attribution_gini`	0.131 $\pm$ 0.011
`mi_xy_mean`	0.053 $\pm$ 0.005
`mi_xy_max`	0.017 $\pm$ 0.004
`disagreement_l2_mean`	0.014 $\pm$ 0.002
`disagreement_l2_vs_lime`	0.007 $\pm$ 0.001
`train_r2`	0.006 $\pm$ 0.001
`mutual_info_max`	0.004 $\pm$ 0.002
`sensitivity`	0.004 $\pm$ 0.001
`local_dispersion`	0.003 $\pm$ 0.001

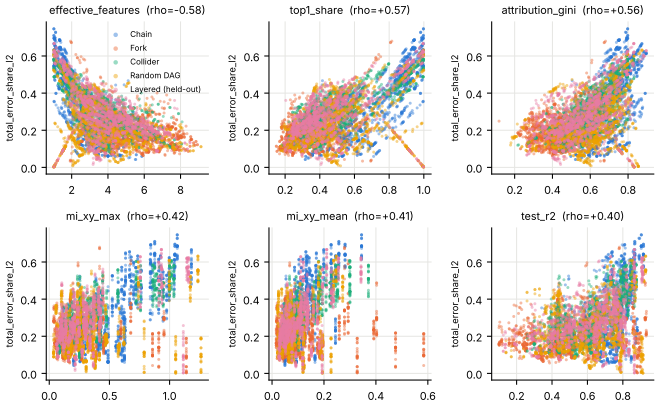


Fig. 6. The six observables most correlated with the share error, one point per explanation, coloured by graph family (Spearman ρ in each title).

G. Ranking SCMs for a fixed explainer

Part of the difference in error comes simply from which explainer is used, and the meta-model can learn this from the explainer name. Table IX removes this effect by measuring performance within each (model, explainer) pair. The meta-model still ranks SCMs by share error with $\rho = 0.45$–0.55 under LOGFO, compared to 0.27–0.47 for the test-error baseline. The difference is largest for LIME and smallest for permutation importance.

IX. DISCUSSION

A. What the results mean

It is already known that SHAP values are not causal effects. Our results measure how large this difference is and where it comes from. The largest part of the error is not caused by the explainer or by the model, but by the fact that SHAP measures something different from the causal effect. This has a practical consequence: using a more accurate SHAP method (more samples, or TreeSHAP instead of KernelSHAP) only reduces the smallest part of the error. To reduce the largest part, one would need a different kind of attribution, such as causal Shapley values [7], [8], and these need the causal graph, which is usually not available.

The second result is more positive. The size of the error is related to signals that can be computed without the graph. The most useful signals are disagreement (between explainers, and between the two models) and fit and concentration (test R^2, how spread the predictions are, how concentrated the attributions are). When explainers or

TABLE IX
SPEARMAN ρ BETWEEN PREDICTED AND ACTUAL SHARE ERROR WITHIN EACH (MODEL, EXPLAINER) PAIR UNDER LOGFO, I.E. RANKING SCMS WHEN THE EXPLANATION METHOD IS FIXED.

Subset	Meta: GBM	Test error
mlp/exact	0.45	0.38
mlp/kernelshap	0.45	0.38
mlp/lime	0.45	0.27
mlp/permutation	0.53	0.44
xgboost/exact	0.47	0.39
xgboost/kernelshap	0.47	0.39
xgboost/lime	0.47	0.30
xgboost/permutation	0.55	0.47
xgboost/treeshap	0.47	0.39

models disagree, it usually means the data does not clearly show which correlated feature should get the credit. Maximum VIF, the signal people check most often, is weak on its own but helps in combination with others.

The meta-model works on unseen graph families because it does not learn the families themselves. It never sees the family name or any other structural information, and in LOGFO the test family is completely removed from training. What it learns is a general link between these signals and the error that is shared by all five families. This is what we would need if the method were used on real data with an unknown causal structure.

B. Limitations

Small synthetic benchmark. All SCMs are small ($d \leq 10$, so that exact Shapley values can be computed), use Gaussian noise and one continuous outcome, and come from five families that we designed. Real data has categorical features, more complex noise and many more features. The meta-model should be tested again before it is used on other data. The planned tests on semi-synthetic datasets (IHDP, Adult Income, California Housing) are left for future work.

Choice of causal effect. We compare against one causal quantity: the total effect of a one-standard-deviation change. Other choices, such as the direct effect only, would give a different estimand gap. The method of splitting the error does not change, but the numbers would.

What the meta-model can do. It predicts how far an explanation is likely to be from the true ranking. It does not say which features are wrong or what the correct explanation is. A high predicted error is a reason to look for domain knowledge or run an experiment, not a correction.

Dependent rows. Rows from the same SCM share the same data. We always group by SCM in evaluation and testing, but each family has only 60 SCMs, so the per-family results are less certain than the overall ones.

Compute cost. Exact Shapley values need $100 \times 50 \times 2^d$ model predictions per seed. The full run took about three hours on two CPU cores. Larger d would need approximate Shapley values even for the reference, which would make it harder to separate the model misfit from the explainer error.

X. CONCLUSION

In this project we tested whether the error of a feature attribution, measured against the true causal effect, can be predicted without knowing the true causes. On a benchmark of 300 synthetic causal models, we found two main results. First, most of the error comes from the definition of SHAP itself: even exact Shapley values of the true function give about a third of the importance to the wrong features compared to the true effect, and using a better explainer does not change this. Second, the parts of the error that come from the trained model and from the explainer can be predicted well from simple signals. A gradient-boosted meta-model predicts the total share error on graph families it has never seen and performs better than all simple baselines. However, the largest part of the error, the estimand gap, is mostly not visible in these signals, and the error in feature ranking is hardly predictable. In practice, this means that such checks can warn a user when an explanation is unreliable because of the model or the explainer, but they cannot replace causal knowledge about whether SHAP answers the causal question at all. The code, data, notebook and all tables are available at https://github.com/stacktrace-bhuvi/datasphere.

CONTRIBUTIONS

B.S.K.: SCM generation, intervention simulation, exact Shapley values of the true function, data formats. M.S.: model training, explainer code, saving attributions. P.L.B.R.: error metrics, error decomposition, error tables. D.S.: observables, meta-models, baselines, evaluation, statistical tests and figures.

REFERENCES

[1] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017.

[2] M. T. Ribeiro, S. Singh, and C. Guestrin, " 'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 1135–1144.

[3] D. Janzing, L. Minorics, and P. Blöbaum, "Feature relevance quantification in explainable AI: A causal problem," in *Proc. 23rd AISTATS*, PMLR 108, 2020, pp. 2907–2916.

[4] S. Krishna, T. Han, A. Gu, J. Pombra, S. Jabbari, S. Wu, and H. Lakkaraju, "The disagreement problem in explainable machine learning: A practitioner's perspective," *arXiv:2202.01602*, 2022.

[5] C. Agarwal, S. Krishna, E. Saxena, M. Pawelczyk, N. Johnson, I. Puri, M. Zitnik, and H. Lakkaraju, "OpenXAI: Towards a transparent evaluation of model explanations," in *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Heskes, E. Sijben, I. G. Bucur, and T. Claassen, "Causal Shapley values: Exploiting causal knowledge to explain individual predictions of complex models," in *Advances in Neural Information Processing Systems 33*, 2020.

[8] C. Frye, C. Rowat, and I. Feige, "Asymmetric Shapley values: Incorporating causal knowledge into model-agnostic explainability," in *Advances in Neural Information Processing Systems 33*, 2020.

[9] D. Alvarez-Melis and T. S. Jaakkola, "On the robustness of interpretability methods," in *ICML Workshop on Human Interpretability in Machine Learning*, 2018.

[10] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge University Press, 2009.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.